The Robot

Fixed Firmware, Two Receiver Boards

All devices are locked to a fixed WiFi channel (channel 1) so they can hear each other without joining a network. Peers are registered explicitly via esp_now_add_peer() using each other's hardcoded MAC address — clean unicast, no broadcast noise.

- Motor receiver — drives the motors from continuous drive packets
- Relay receiver — switches lights/accessories from one-shot events

Telling Packets Apart — By Length

Each function gets its own small, fixed-size, packed C struct. The receiver tells packet types apart by length — a zero-overhead form of message typing that needs no header byte.

```
typedef struct __attribute__((packed)) {
  uint16_t x;     // joystick X, 0-4095
  uint16_t y;     // joystick Y, 0-4095
  uint32_t seq;   // rolling sequence #
  uint32_t ms;    // sender timestamp
} JoyPacket;    // 12 bytes — drive
```

```
typedef struct __attribute__((packed)) {
  uint8_t  state; // 0 = off, 1 = on
  uint32_t seq;
} LightPacket;  // 5 bytes — accessory event
```

Talking Back: the Heartbeat

The motor receiver sends a periodic heartbeat packet back to any listening controller, roughly every 500ms — RSSI, relay state, uptime. A screen-equipped controller renders this as a live HUD; a simpler controller just never asks for it. The robot doesn't need to know or care which is connected.

Reliability, Handled at the Application Layer

- Continuous streams (drive) are self-healing — a dropped packet is superseded 5ms later; a receiver-side timeout is a fail-safe stop
- One-shot events (light toggles) are made reliable by resending a few times in a burst rather than waiting on an ACK